

How does containerisation work?

Let's go step-by-step through the container lifecycle.

Writing a Dockerfile: A Dockerfile is a blueprint to build your container image.

Example: Dockerfile for a Python App:

```
dockerfile
# Use an official Python runtime as a base image
FROM python:3.10

# Set the working directory
WORKDIR /app
# Install dependencies
RUN pip install -r requirements.txt
# Run the application
CMD ["python", "app.py"]
```

Building the container image: Use the Docker CLI to build the image:

```
docker build -t my-python-app .
```

This generates a container image named *my-python-app*.

Running a container: You can start a container from the image:

```
docker run -d -p 5000:5000 my-python-app
```

This runs the app in the background and maps port 5000 of the host to the container.

Container registries: To share the image, use:

```
docker tag my-python-app username/my-python-app
docker push username/my-python-app
```

Now others can pull it with:

```
docker pull username/my-python-app
```

Container orchestration: Once applications grow, you'll need to manage multiple containers. Use orchestrators, of which Kubernetes is the most popular. With it, you can:

- Deploy and scale containers automatically
- Monitor container health
- Manage networking between containers
- Load balance and update containers with zero downtime

Real-world use cases

Microservices architecture: Each microservice can be containerised and deployed independently. For example, a travel app can have separate containers for booking, payments, and notifications.

Hybrid cloud deployments: Containers can run on AWS, Azure, GCP, or on-premises. This makes it easy to move workloads across providers.

Continuous integration/Continuous delivery (CI/CD): Containers allow for repeatable builds and tests, making them ideal for automation pipelines.

Development sandboxes: Developers can use containers to replicate production-like environments on their local machines without conflicts.

Advantages and disadvantages of containers

Advantages

- Rapid deployment and scaling
- Efficient use of system resources
- Simplified dependency management
- Faster development cycles
- Easy rollbacks and version control

Disadvantages

- Persistent storage is complex
- Security needs careful handling
- Learning curve for orchestration (e.g., Kubernetes)
- Container sprawl if not managed well

Popular tools in the container ecosystem

Tool	Purpose
Docker	Build and run containers
Podman	Docker-compatible, daemonless container engine
Kubernetes	Container orchestration
Helm	Kubernetes package manager
Docker Compose	Define multi-container apps
Harbor	Secure container registry
Prometheus + Grafana	Monitoring container environments

Containerisation bridges the gap between development and operations, driving agility and innovation. As organisations continue to embrace cloud-native architectures, mastering containers remains a critical skill for the future of DevOps. 

 By: Mohit Raj

The author is a CEH and ECSA, and has worked at IBM, Teramatrix and Sapient. He is the author of the books titled 'Python Penetration Testing Essential', 'Python: Penetration Testing For Developers', and 'Learn Python in 7 Days'.